

Indianapolis Restaurant Recommender

Shanaya Mehra – DS 210 Final Project

A. Project Overview

The aim of this project is to answer a simple question: how can we recommend Indianapolis restaurants to a user by combining their own Yelp ratings with the tastes of diners who rate in a similar way? This idea grew out of a personal need. After relocating to Boston for university I realised firsthand how overwhelming it is to find good places to eat in an unfamiliar city. Since my older brother now studies in Indianapolis and I'll be visiting him soon, I wanted a tool that could instantly surface restaurants we are likely to enjoy, guided not just by star averages but by people whose preferences mirror our own.

To support this recommender, I drew on the Yelp Open Dataset from Kaggle. From the multi-gigabyte corpus, I isolated only Indianapolis entries, producing a manageable yet meaningful working set of roughly 1200 restaurants, 25000 reviews, and 5000 users. These trimmed JSONL files are small enough to live in the repository while still large enough to demonstrate realistic collaborative-filtering behaviour.

Link: <https://www.kaggle.com/datasets/yelp-dataset/yelp-dataset>

B. Data Processing

The raw Yelp Academic Dataset ships as several multi-gigabyte, newline-delimited JSON files, but the recommender needs information from only three of them (business, review, and user). To pare those files down, I wrote three concise Python scripts. The first script processes the business file, keeps rows whose city is Indianapolis and whose category string contains the word Restaurant, and writes out just the business_id plus a human-readable name. The second script scans the review file, retaining only reviews whose business_id survived that first filter, and emits a minimal record of user_id, business_id, and stars. The final script loads the user file and preserves only users referenced in the cleaned reviews, storing each user's user_id together with their public display name. Each cleaner prints how many records were written and shows a five-row preview so I can spot-check the output before passing the data to Rust.

Within Rust, `src/data.rs` provides a generic `load_jsonl` function that wraps a `BufReader` around each cleaned file and stream-parses it with `Serde` into strongly typed vectors. Thin convenience wrappers like `load_restaurants`, `load_reviews`, and `load_users` invoke this helper so that the rest of the program can work with well-typed collections.

C. Code Structure

The codebase is deliberately divided into a handful of small, focused modules. `src/data.rs` concentrates on data ingestion- it defines the record structs `Restaurant`, `Review`, and `User`, and exposes the trio of loaders `load_restaurants`, `load_reviews`, and `load_users`. All three call a generic helper `load_jsonl<T>` that streams a newline-delimited JSON file through a `BufReader` and uses `Serde` to deserialize each line into a strongly typed object, returning a `Vec<T>`. `src/recommend.rs` holds the core algorithm. Its primary function, `recommend_for`, accepts a target `user_id` plus slices of `Restaurant`, `Review`, and `User`, and returns a `Vec<Recommendation>` (a struct bundling a `Restaurant`, its `business_id`, and a computed score). `src/main.rs` provides the CLI front-end, wiring `Clap`-parsed arguments to the loaders and then to `recommend_for` before formatting the top-five list for display. `src/lib.rs` simply re-exports data and `recommend` so that both `main.rs` and the unit tests can import them cleanly. Finally, `tests/mod.rs` exercises `recommend_for` in isolation, ensuring algorithmic correctness independent of CLI concerns.

At runtime the workflow is straightforward- `main.rs` calls the loaders in `data.rs`, receiving typed vectors of restaurants, reviews, and users. These references are fed into `recommend_for`, which first aggregates each user's ratings into a hash-map, then computes dot-product similarities between the target user and every other user over co-rated restaurants. Using those similarity weights it predicts scores for restaurants the target user has not yet tried, normalises by total weight, sorts the candidates by descending score (breaking ties lexicographically by `business_id`), and returns the five best. `main.rs` then prints those results, while the unit-test module verifies that the function produces the expected output for edge cases and simple toy scenarios.

The directory tree looks like this:

```
finalproject/
├── .gitignore
├── Cargo.lock
├── Cargo.toml
├── README.md
├── written_report.pdf          # complete write-up in PDF form
├── output/
│   └── output.png             # sample CLI screenshot
├── src/
│   ├── dataset/
│   │   ├── cleaned_indianapolis_restaurants.json
│   │   ├── cleaned_indianapolis_reviews.json
│   │   └── cleaned_indianapolis_users.json
│   ├── data_cleaning.ipynb    # Jupyter notebook for Python cleaning
│   ├── data.rs                # JSONL loaders & record structs
│   ├── lib.rs                 # module re-exports
│   ├── main.rs                # CLI parsing & result printing
│   └── recommend.rs           # collaborative-filtering algorithm
├── tests/
│   └── mod.rs                 # unit tests for recommend_for()
└──
```

D. Tests

To make sure the heart of the recommender behaves, I wrote two focused unit tests in `tests/mod.rs`. The first, `no_recommend_when_no_similar_users`, confirms that when the dataset contains only a single user, the system sensibly returns no recommendations—crucial because emitting random restaurants in the absence of neighbours would undermine user trust. The second test constructs a synthetic two-user, two-restaurant scenario in which both users rate one restaurant at five stars and only the neighbour rates a second restaurant at three stars. It verifies that the second restaurant is recommended to the target user with a predicted score of exactly 3.0, proving that both the dot-product similarity and weighted-average scoring logic are implemented correctly. Together these tests guard against the two most damaging failure modes: recommending when no evidence exists and mis-computing scores when evidence does exist.

The cargo test output was:

```
$ cargo test
Finished `test` profile [unoptimized + debuginfo] target(s) in 2.38s
Running unittests src/lib.rs (target/debug/deps/finalproject-f5b54dd9e1c90d22)

running 0 tests
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Running unittests src/main.rs (target/debug/deps/finalproject-c4714d8c90dc01c5)

running 0 tests
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Running tests/mod.rs (target/debug/deps/mod-b23cf62a1659833e)

running 2 tests
test no_recommend_when_no_similar_users ... ok
test recommend_simple_case ... ok
test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

Doc-tests finalproject

running 0 tests
test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

[/opt/app-root/src/finalproject]
```

E. Results

When the program is invoked with no arguments it automatically loads the cleaned JSONL samples, selects the first reviewer in the file (user ID `ZbqSHbgCjzVAqaa7NKWn5A`, display name “Small”), and prints the five highest-scoring restaurants that Small has not yet rated. In the present dataset all five suggested venues tie at 5.000, and, because the algorithm breaks ties lexicographically by `business_id`, the list is emitted in the same order every run. This deterministic behaviour is important in the project context as it demonstrates that the recommender can deliver stable, reproducible results even when scores collide, and it prevents the user-confusing shuffle that would occur if we relied on hash-map iteration order.

The cargo run output was:

```
$ cargo run
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.03s
  Running `target/debug/finalproject`
Top 5 recommendations for Small (user ID: ZbqSHbgCjzVAqaa7NKWn5A):
1. Wings and Seafood (business ID: -2vEnMoexef2-rWy-FMWvw) - score = 5.000
2. Taco Bell (business ID: -A-5spZpPMjujSnG-0ag7w) - score = 5.000
3. Droopy's BBQ (business ID: -H6a4mIT5GbQiiWZz2lu0w) - score = 5.000
4. Sarita's Pollos Charbroiled Cuisine (business ID: -LoVysdztSNMiUeHCw8Ldg) - score = 5.000
5. MJ's Backyard BBQ and Catering (business ID: -nJid5B14dYu1wPtMp149g) - score = 5.000
[/opt/app-root/src/finalproject]
```

F. Usage Instructions

After cloning the repository, a single command—`cargo build --release`, builds the binary. Typing `cargo run` with no further flags is enough to see a full recommendation list in about one-tenth of a second on a mid-range laptop. Advanced users can override any of the three input paths or specify a different Yelp user with the `--restaurants`, `--reviews`, `--users`, and `--user-id` options.

```
[/opt/app-root/src/finalproject]
$ cargo run -- \
  --restaurants src/dataset/cleaned_indianapolis_restaurants.json \
  --reviews      src/dataset/cleaned_indianapolis_reviews.json \
  --users        src/dataset/cleaned_indianapolis_users.json \
  --user-id      ZbqSHbgCjzVAqaa7NKWn5A
```

G. AI-Assistance Disclosure and Citations

I consulted ChatGPT for two purposes. First, commit workflow. The model explained that each commit should encapsulate one coherent idea such as “add data loader,” “implement recommender,” or “add tests” and that messages should be imperative and descriptive (e.g., “Add load_reviews parser”). Following this advice I learned to stage only the relevant files, commit frequently, and write messages that let future readers trace the rationale behind every change.

Second, README structure. ChatGPT provided examples of clear README layouts, which I distilled into six short sections (Overview, Methodology, Structure, Usage, Testing, and Conclusion) so newcomers can understand, build, and run the project in minutes.